DESIGN / CONFIGURATION / AI ASSISTANCE

FutureUI Studio

User guide

From your first menu to AI-assisted creation

Illustrated steps, practice files and restore points

WINDOWS x64 / VERSION 0.1.2

ENGLISH / Step-by-step edition

FOTIA / 2026-10-16

github.com/Ti-Avanti/FutureUI-Studio

Start here. Return when you need it.

Click any entry to jump. Chinese UI labels are retained with English explanations. Both editions use the same page numbers.

Understand the complete workflow 03

Install and launch Studio 04

Create, open and organize a project 05

Find your way around the workspace 06

Prepare a usable first menu 07

Import the first menu into your project 08

Select a component and change a button 09

Work with layout, templates and pixel sizes 10

Use the canvas and YAML together 11

Give a button an action 12

Preview player data and translations 13

Import pixel art and use it in a menu 14

Inspect hover, selection and frame animation 15

Import multiple configuration files 16

Choose merge, replace or keep 17

Resolve conflicts one value at a time 18

Export current, selected or modified files 19

Return to an operation in the current session 20

Use restore points after reopening Studio 21

Connect your own model service 22

Use the Anthropic Messages protocol 23

Import, update and enable Skills 24

Give Agent precise context and review its work 25

Manage sessions, context usage and compression 26

Deliver a menu to a Minecraft server 27

Update after receiving a notification 28

Complete an edit-and-deliver exercise 29

Editor, import and restore troubleshooting 30

Agent, updates and server connection issues 31

Keyboard shortcuts and further help 32

BEFORE YOU START

Understand the complete workflow

This guide covers **FutureUI Studio 0.1.2 for Windows x64**. You can follow it without previous YAML experience. Practice in a local project first, then deliver the reviewed configuration to your server.

01	02	03	04	05
Create project	Edit menus	Preview / restore	Export	Verify in game

Prepare	Why you need it
A Windows x64 PC	Runs the installer or portable build. You do not need to install Node.js just to use the editor.
A local configuration folder	Stores menus, templates, translations and assets. The first launch can create an example project.
A FutureUI game server (for deployment)	Runs menus, real purchases, PAPI placeholders and third-party integrations.
A model service (optional)	Base URL, API key and model ID are only needed for Agent features.

01 First time: read installation, projects and the workspace tour; then complete the first-menu exercise.

02 Already have configurations? Start with batch import, conflict handling and quick export.

03 Want AI assistance? Configure a model and Skills, then learn the proposal-review workflow.

04 Made a mistake? Open 操作与回退 (Operations and restore), inspect the diff, then select a restore point.

> **Remember this first**
>
> **Editing, saving, exporting and reloading a server are different operations.** The local canvas is not a Minecraft client. A correct preview does not prove that server-side business logic works.

01 / INSTALLATION

Install and launch Studio

01 Open this project's GitHub Releases and select **v0.1.2**. Download FutureUI-Studio-Setup-0.1.2.exe.

02 Run the installer, choose an installation directory and complete the wizard. Launch FutureUI Studio from the desktop or Start menu shortcut.

03 Wait for the example project to load. You should see menu files on the left, a canvas in the center and a save status at the bottom.

04 You can edit without AI. An API key is not required to use the editor.

05 Use 文件 > 新建工作工程 (File > New working project) for practice. Avoid using your live server directory for your first exercise.

Download	Use it for
Setup installer	Daily use. Supports update notifications, manual downloads and relaunch after installation.
Portable build	Temporary or portable use. Install Setup once to enable in-app installation of updates.
.blockmap / latest.yml	Used by the updater. Users do not open or run these files themselves.

> **Success checkpoint**
> The window is titled FutureUI Studio and shows a project name. Use 帮助 > 关于 FutureUI Studio (Help > About) to inspect the application version. The bottom label “FutureUI 1.0.0” describes the configuration capability baseline, not the Studio application version.

Release page: FutureUI Studio Releases

02 / PROJECTS

Create, open and organize a project

01 Create: choose 文件 > 新建工作工程 (File > New working project), then choose a parent folder. Studio creates FutureUI-工作工程 there and copies the examples. If that folder already exists, choose another parent.

02 Open: press Ctrl+O and select the configuration root containing a menus directory. Do not select a single YAML file or a subfolder inside menus.

03 You may also open a plugin source folder containing src/main/resources/menus; Studio recognizes the resource directory.

04 Save changes you want to keep before switching projects or rereading disk files. Read the discard-changes prompt before accepting it.

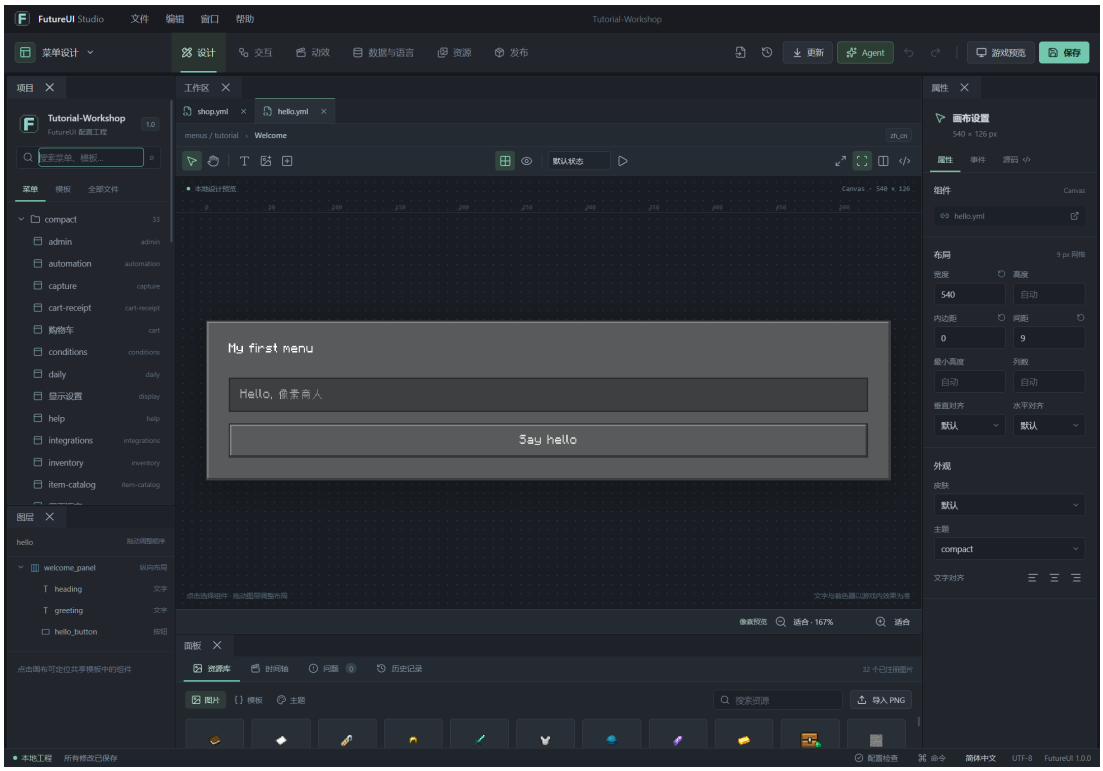
Path	Contents
menus/	Menu pages.
templates/	Shared components referenced by menus.
languages/zh_cn/, languages/en_us/	Localized text.
shops/, functions/, rules/, hud/	Products, reusable flows, rules and HUD configuration.
assets/	Image definitions, themes, image files and shaders.
config.yml	The main configuration in the project.
.futureui-studio/	Local backup revisions and saved restore points, not server menu files.

Cannot find a file?

The left panel initially filters menus. Switch to 模板 (Templates) or 全部文件 (All files), then search by filename or path. A file path and a menu's id are not the same thing.

03 / WORKSPACE

Find your way around the workspace



Actual workspace with the tutorial project. Zoom the PDF to inspect details; the menu shown is the exercise used in this guide.

Area	What you do there
Top workspace tabs	Switch between design, interaction, motion, data/languages, assets and publishing.
项目 (Project), top left	Search and open YAML files; keep several file tabs open.
图层 (Layers), bottom left	Select components and drag layers to reorder them within the same file.
工作区 (Workspace), center	Use canvas, YAML or split view; inspect hover states and simulate interactions.
属性 (Inspector), right	Edit size, spacing, skin, text and other properties of the selected component.
面板 (Panel), bottom	Assets, timeline, issues and history.

If panels become misplaced

Choose 窗口 > 重置工作区布局 (Window > Reset workspace layout). Press F outside text fields to focus the canvas; press it again to restore. Open Agent from the top bar or with Ctrl+J.

04 / YOUR FIRST MENU

Prepare a usable first menu

Download and extract the tutorial examples ZIP. This file is 01-first-menu/menus/tutorial/hello.yml. Alternatively, save the complete content below as UTF-8 without BOM. Do not wrap it in an extra menus: key.

```
id: tutorial/hello
title: '<!i>Welcome'
renderer: canvas
layout:
  type: column
  width: 540
  padding: 0
  gap: 9
  theme: compact
  exit-button: true
  exit-width: 126
components:
  - id: welcome_panel
    type: column
    skin: panel
    padding: 18
    gap: 9
    children:
      - id: heading
        type: text
        text: '<!i><white>My first menu'
        height: 18
      - id: greeting
        type: text
        text: '<!i><gray>Hello, {player.name}'
        height: 27
        skin: readout
        padding: 9
      - id: hello_button
        type: button
        text: '<!i>Say hello'
        height: 27
        skin: button
        actions:
          - type: message
            text: '<!i>Hello from FutureUI!'
```

What this example does

It displays a heading, a player name and a message button using the example project's compact theme. It performs no transaction. 02-merge-update is a merge patch for a later exercise; do not replace this complete page with that patch.

04 / YOUR FIRST MENU

Import the first menu into your project

01 Open a working project. Press Ctrl+Shift+I, or click the top-bar 配置导入与导出 (Configuration import/export) icon.

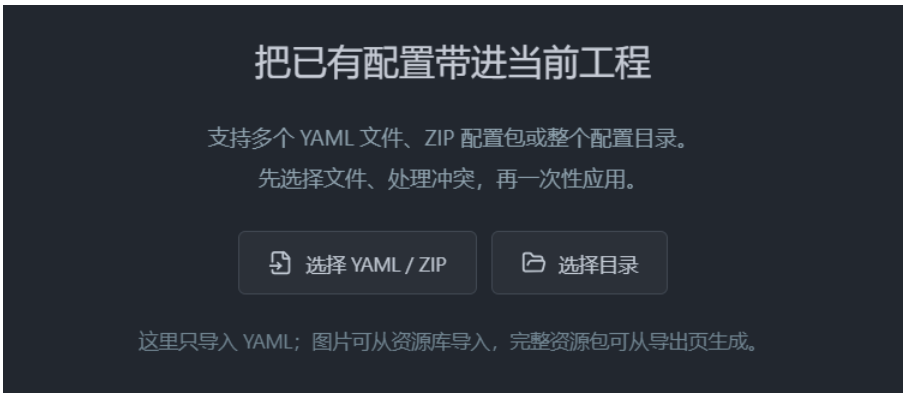
02 Choose 导入与合并 > 选择目录 (Import and merge > Choose directory), then select the extracted 01-first-menu folder. You can also choose hello.yml individually.

03 Ensure the file is checked. Its target should be menus/tutorial/hello.yml; correct the target field manually if it was not detected.

04 The diff should show 当前不存在 (Does not currently exist) and the incoming content. Click 应用导入 (Apply import). This changes the editor project, not the files on disk yet.

05 Press Ctrl+S to save. Search for tutorial/hello in the Project panel and open it.

06 Confirm the canvas shows My first menu, Hello and Say hello. The status bar should read 所有修改已保存 (All changes saved).



Import entry point. For this exercise select only 01-first-menu; use the other samples in later chapters.

> **Practical ways to add a menu**
> In 0.1.2, create new files through configuration import or an Agent proposal. 新建工作工程 creates a whole project, not a single menu.

05 / VISUAL EDITING

Select a component and change a button

01 Open tutorial/hello, then click hello_button in Layers. You can also click the canvas button while using the selection tool.

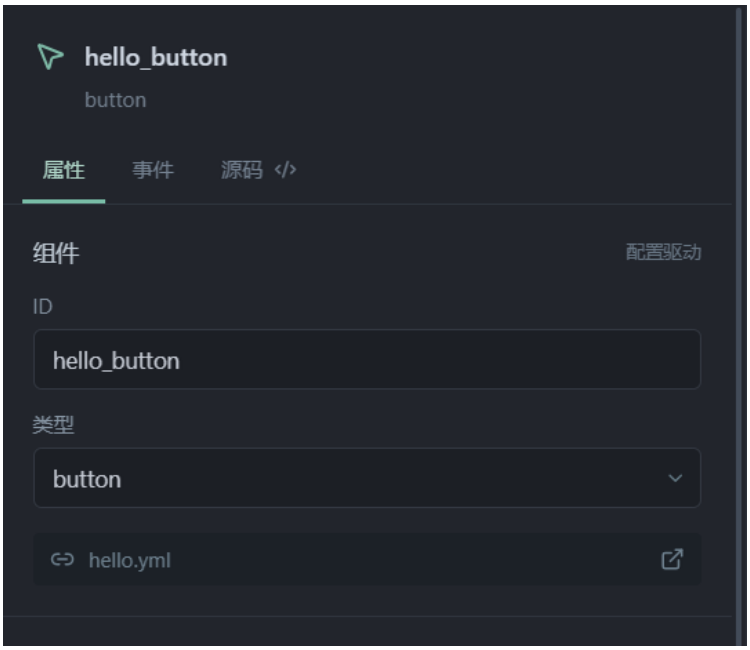
02 The Inspector heading should show hello_button. If it still shows 画布设置 (Canvas settings), you have not selected a component.

03 Enter new text under 文字内容 (Text content). Click outside the text box to commit it; other properties commit through their respective controls.

04 Change height to 36, keeping the button skin. Observe the matching changes in the canvas and YAML.

05 Try Ctrl+Z or the top Undo button, then redo. Press Ctrl+S when satisfied.

06 For similar buttons, press Ctrl+D outside a text input to duplicate the selection. Use Delete to remove the selected component.



Component properties. Confirm the selected object before changing size or text.

> **Editing tip**
> Dragging the canvas pans the view; it does not freely reposition controls. Containers such as row, column and grid, together with size and spacing, determine layout.

05 / VISUAL EDITING

Work with layout, templates and pixel sizes

Setting	Meaning and useful starting point
row / column / grid	Horizontal, vertical and grid containers. Decide the structure before adding components.
width / height	Game GUI pixel units, not monitor pixels. Preview zoom does not change configuration dimensions.
padding / gap	Inside padding and spacing between children. Values such as 9, 18 and 27 make a consistent starting grid.
align / vertical-align	Horizontal and vertical alignment. Text alignment is separately controlled by text-align.
skin	Appearance such as panel, button, selected or readout. Available skins depend on the project's theme.
template	The shared component source. Editing the template affects pages that reference it.

01 Select a container, then use 添加文字 / 添加图片 / 添加按钮 (Add text/image/button). New components go into the selected container, or usually into the page's root component list if no container is selected.

02 Drag layers within the same file to change order. Direct cross-file moves are blocked; copy the configuration and check references instead.

03 For a template instance, check 来源文件 (Source file). Confirm in YAML whether you are editing the shared template or the current instance.

04 Use 适合 (Fit) to inspect the whole menu, then 100% or higher zoom to inspect pixel edges and text spacing.

> **After a layout change**
> Check long text, hover states and smaller windows. The real client is authoritative for text, fonts and shaders; avoid masking overflow with oversized fixed dimensions.

05 / VISUAL EDITING

Use the canvas and YAML together

01 Click 画布与源码并排 (Canvas and YAML side by side). Inspect the result while editing YAML. Switch to YAML 源码 (YAML source) when you want the code view alone.

02 Find the text under hello_button and edit the quoted string. Keep indentation and the component ID intact, then observe the preview.

03 Use the editor's completions and validation messages when working with fields. The bottom 问题 (Problems) panel lists configuration issues you can locate.

04 Fix YAML errors first. Before saving, review Problems and run 配置检查 (Configuration check). The canvas alone is not a complete validator.

05 To review what changed, open 操作与回退 (Operations and restore), select a node and inspect its diff. Save when satisfied.

Syntax	Explanation
Two-space indentation	Use spaces to represent nesting. Do not mix tabs and spaces.
- id: hello_button	A component in a list. Keep IDs unique in that component list.
text: '<!i>Say hello'	Quote strings containing MiniMessage tags. <!i> disables default italics.
text: '@tutorial.greeting'	A translation-key reference. Keep quotes, particularly around strings beginning with @.
true / false, 36	Booleans and numbers usually need no quotes. Match the field's actual type.

> **Two kinds of undo**
> Inside a text editor, Ctrl+Z normally undoes text edits. Outside text fields, workspace undo handles component edits or grouped operations such as imports. For cross-file restores, use Operations and restore to inspect the diff.

06 / INTERACTION

Give a button an action

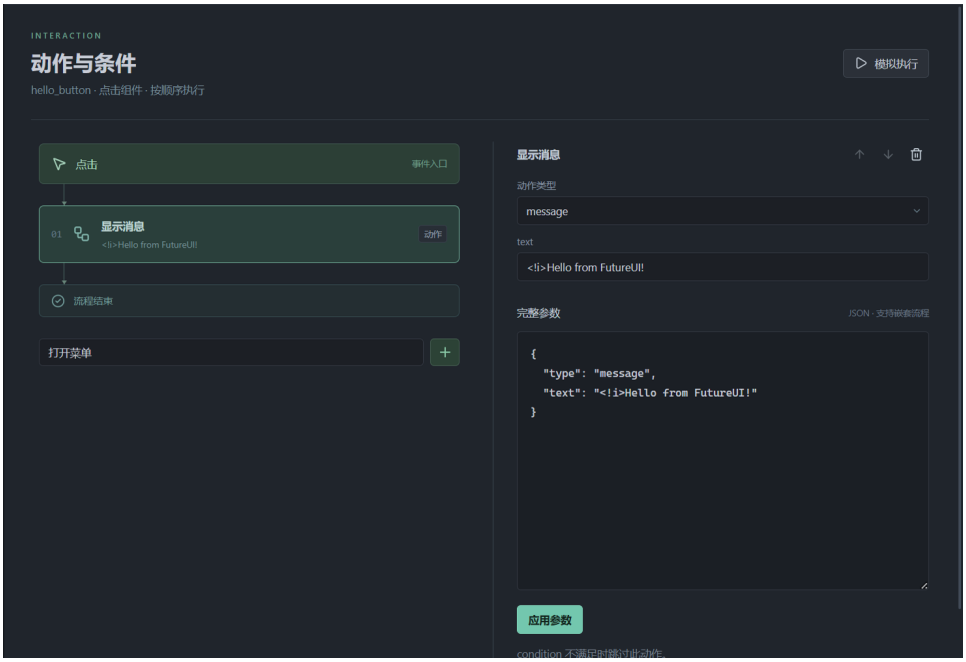
01 Select hello_button and open 交互 (Interaction), or the Inspector's 事件 (Events) tab. With a component selected you edit its actions; otherwise you edit the menu's on-open flow.

02 The tutorial button already has a message action. Select it, change text and click 应用参数 (Apply parameters). This parameter editor uses JSON: quote keys and strings with double quotes.

03 Add actions using the action-type selector and arrange them in execution order. For navigation use open-menu and a menu ID such as compact/main.

04 Enable 交互模拟 (Interaction simulation), click the canvas button and observe local messages or variable changes. Exit simulation to continue designing.

05 Verify real purchases, commands and integrations after deployment to a server. The simulator does not charge players or execute server commands.



Actual action panel showing the button's message action.

Concept	Purpose
condition	Skip that action when its condition fails.
require	Stop the current flow when a prerequisite fails.
if / then / else	Choose between branches based on a condition.
State versus action	Selected and hovered are visual states. Availability also depends on conditions and the actual flow.

07 / DATA AND LANGUAGES

Preview player data and translations

01 Open 数据与语言 (Data and languages). Preview variables are JSON. Change player.name to a longer name, then click 应用预览变量 (Apply preview variables).

02 Try presets such as low balance, long name and empty list. These affect local previews only; they do not modify server player data.

03 Import and save the tutorial folder 03-language-entries. It contains languages/zh_cn/tutorial.yml and languages/en_us/tutorial.yml.

04 Change the greeting component's text to @tutorial.greeting, keeping quotes in YAML. Switch 简体中文 / English at the bottom to compare translations.

05 To edit products, choose a shop file in the product-data area, change a name, price or bundle amount, and save the corresponding YAML.



Preview variables and language data help inspect edge cases.

> **Bundles are not individual items**
> At 16 items and 8 coins per bundle, buying 3 bundles means 48 items and 24 coins. Balance, inventory capacity and purchase rules are ultimately checked by the server.

The bottom language selector changes the **menu preview language**, not Studio's interface language. Offline previews do not query live PAPI or third-party plugins; provide representative values or verify in game.

08 / ASSETS

Import pixel art and use it in a menu

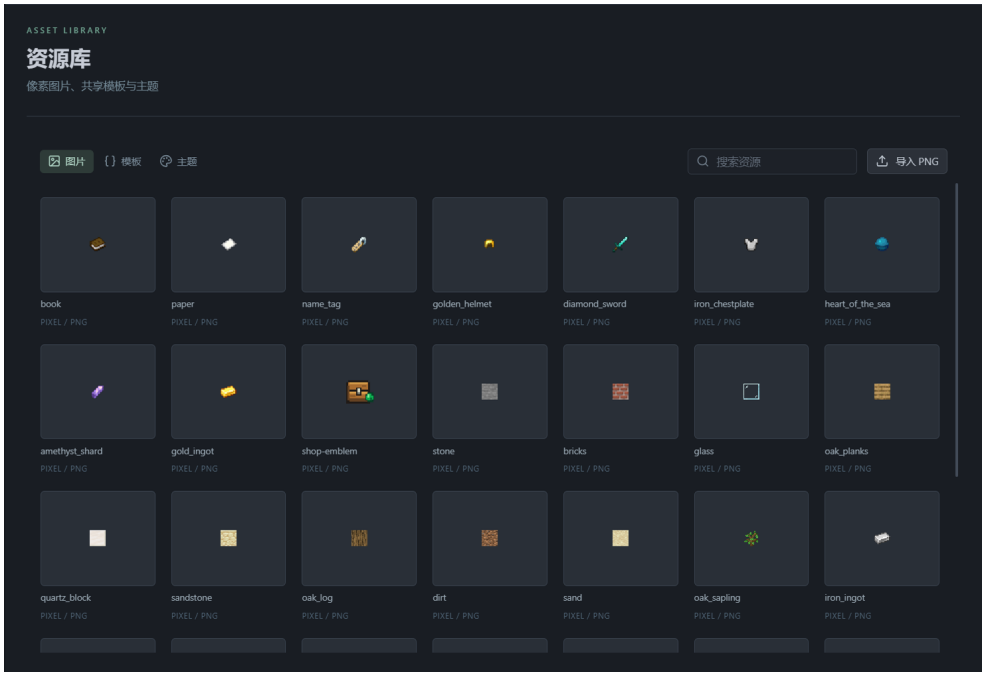
01 Open 资源 (Assets), or 面板 > 资源库 > 图片 (Panel > Asset library > Images). Click 导入 PNG (Import PNG). Each image must be no larger than 8 MB.

02 The image is copied immediately into assets/textures/studio. Its definition is added to assets/images/studio.yml. Press Ctrl+S to save that registration.

03 Select an image component and double-click an asset card to replace its image. Without an image component selected, double-clicking inserts a new one.

04 Adjust the display size in the Inspector. Inspect pixels at 100% or higher zoom. Integer scale factors help keep pixel art, small text and lines crisp.

05 To have Agent use an image, click its 发送给 Agent (Send to Agent) control or drag the asset into the Agent composer.



Assets can be inserted into the canvas or attached as Agent context.

> **Assets and YAML are saved differently**
> An imported PNG already exists on disk. Undoing a YAML operation does not delete it, and saved restore points cover YAML only. Back up asset folders for a complete project and include assets when exporting a ZIP.

New image registrations default to canvas-height 64, height 64 and ascent 56. Adjust for the asset rather than stretching every image to the same ratio. For missing images, check the image ID, its definition and the physical path.

09 / STATES AND MOTION

Inspect hover, selection and frame animation

01 Select a button and cycle through normal, hovered, selected and disabled states above the canvas. Confirm that interactive and unavailable controls are visually distinct.

02 Open 动效 (Motion) and select the component to animate. The timeline configures frame-based changes to images or skins.

03 Add a frame, select it and change its image or skin. New frames can inherit the previous frame, ready for individual edits.

04 Set interval-ticks and loop. One tick is 50 ms, so 4 ticks is 0.2 seconds. Enable loop for repeating animation.

05 Preview with the timeline playback controls, then save. Verify the assets and shaders in game as well.

Desired result	Suggested approach
Brighter button on hover	Configure the theme's hover appearance and inspect the hovered state.
Cycling icons	Prepare multiple pixel images and reference them in animation frames.
A green state after selection	Use variables or conditions for the selected state and actions to update the data.
Rotation or complex 3D effects	Do not invent unsupported fields. Use the plugin's implemented capabilities and actual asset approach.

State preview does not create logic

Switching the preview to selected only inspects appearance. It does not automatically implement multiple selection, purchases or persistence. Configure behavior in Interaction.

10 / IMPORT

Import multiple configuration files

01 Press Ctrl+Shift+I. Use 选择 YAML / ZIP (Choose YAML/ZIP) for files or archives, and 选择目录 (Choose directory) for folders.

02 After scanning, review the file list and uncheck anything you do not need. ZIP and directory imports read YAML only; other files are counted as skipped.

03 Check each 目标路径 (Target path). It is relative to the project root, for example menus/tutorial/hello.yml. Do not enter absolute paths or .. segments.

04 For existing files choose merge, replace or keep, then review the diff. Resolve blocking errors such as duplicate YAML keys, target collisions or missing references.

05 Click 应用导入 (Apply import). The selected files become one grouped operation. Inspect the canvas and Problems panel before pressing Ctrl+S.

Limit	What to do
8 MB per YAML; 64 MB total	Split oversized configuration packages.
Up to 4,000 configuration files	Import functional folders in smaller batches.
Up to 12,000 ZIP entries	Keep only relevant configuration and directory structure.
UTF-8 content	Import removes a UTF-8 BOM, but invalid encodings must be converted first.

YAML import is not a complete asset import

Import images through the asset library. Existing resource folders can be copied into the appropriate project location and then reread. YAML import alone does not supply missing PNGs, shaders or third-party resources.

10 / IMPORT

Choose merge, replace or keep

Mode	What it does	Use it when
合并字段 (Merge)	Recursively merges maps and retains untouched fields; you choose conflicting values.	Applying a partial patch while preserving existing configuration.
替换文件 (Replace)	Replaces the entire file; old fields absent from the incoming file disappear.	Importing a reviewed, complete file.
保留现有 (Keep)	Leaves the existing file unchanged.	Skipping an update to this existing file.

If every list entry has a unique string ID, merge matches entries by ID: matching entries merge fields and new IDs append. Existing order is retained. Lists without unique IDs, including many action lists, are treated as **whole values**, not blindly appended.

01 Save the first menu, then import the tutorial's 02-merge-update directory.

02 Confirm the target is menus/tutorial/hello.yml and choose 合并字段 (Merge).

03 Expand 2 处不同值 (2 different values), then choose which text to use for heading and hello_button.

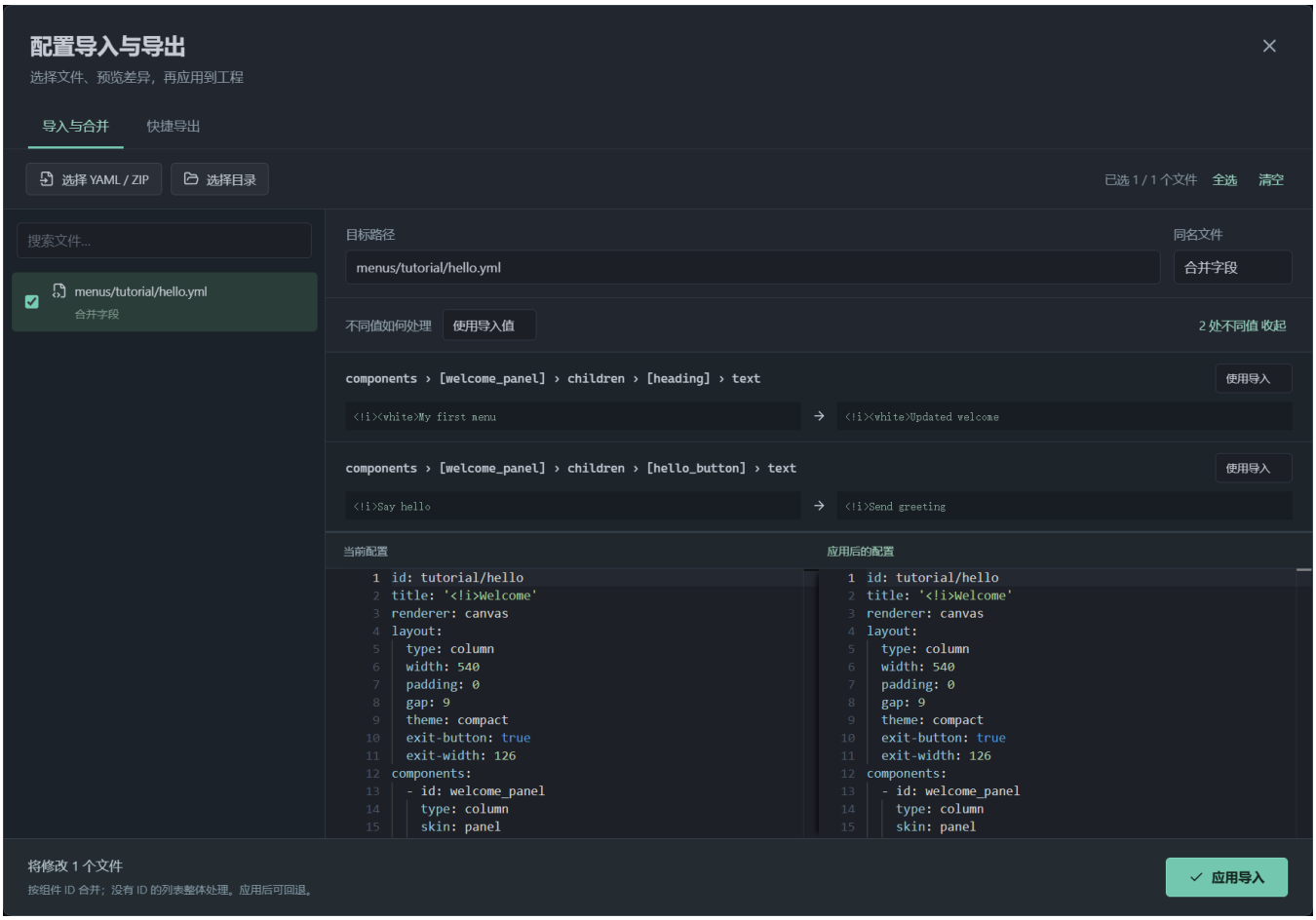
04 The result should retain greeting and add close_button. If the full layout disappears, check that you did not select Replace.

05 Apply, inspect both buttons and save. A single workspace undo should revert the whole import batch.

> **Clearing versus deleting fields**
> Omitting a field from a patch preserves the old field during merge. To intentionally delete old structure, edit the target YAML directly or replace it with a complete file after reviewing the diff.

10 / IMPORT

Resolve conflicts one value at a time



The left side is current content; the right side is the proposed result. Resolve individual conflicts above the diff.

01 Set the default under 不同值如何处理 (How to handle different values). Keep current protects existing settings; use incoming applies updated patch values.

02 Expand the conflict list. A segment such as [heading] refers to a component ID, not a line number. Confirm the intended component.

03 Override individual fields with 保留当前 / 使用导入 (Keep current / Use incoming). Scroll the diff to inspect changes and additions.

04 Check for unintended duplicate actions, missing language references and incorrect targets. Apply, inspect the canvas and save.

> **A conflict does not mean import failed**
>
> Different values can be resolved by choice. Duplicate YAML keys, unsafe paths and duplicate targets are blocking issues. Read the indicated file and location, fix it and select the import again.

11 / EXPORT

Export current, selected or modified files

01 Press Ctrl+Shift+E to open 快捷导出 (Quick export). Choose 当前文件 (Current file), 全部配置 (All configuration) or 未保存的修改 (Unsaved changes).

02 Review the checked paths. One YAML can be exported directly as YAML; use ZIP for multiple configurations to preserve their directory structure.

03 To deliver images and other assets, select ZIP and enable asset inclusion. For a small configuration patch, avoid unintentionally including the entire asset directory.

04 Click 导出所选配置 (Export selected configuration) and choose a destination outside the project, such as a delivery folder on your desktop.

05 Extract and inspect the paths and YAML. Export neither uploads to a server nor marks editor changes as saved.



Choose the exact export scope for configuration patches, page delivery or examples.

Export uses the current editor contents

Unsaved edits are included. Deleted files are not represented as deletion instructions in the ZIP; the recipient must handle removals explicitly. Editor keys, conversations and history are excluded, but passwords or other values you placed in YAML remain in that YAML.

12 / RESTORE

Return to an operation in the current session

01 Press Ctrl+Alt+H or the top 操作节点与回退 (Operation nodes and restore) button. Select 本次操作 (Current session).

02 Select the desired node on the left. It represents the project after that operation. 本次打开工程 is the starting state.

03 Inspect added, modified and deleted files on the right. Switch the diff file to check the full scope.

04 Click 回到此节点 (Return to this node), press Esc to close the dialog, then inspect the canvas. Disk files remain unchanged until you confirm and press Ctrl+S.

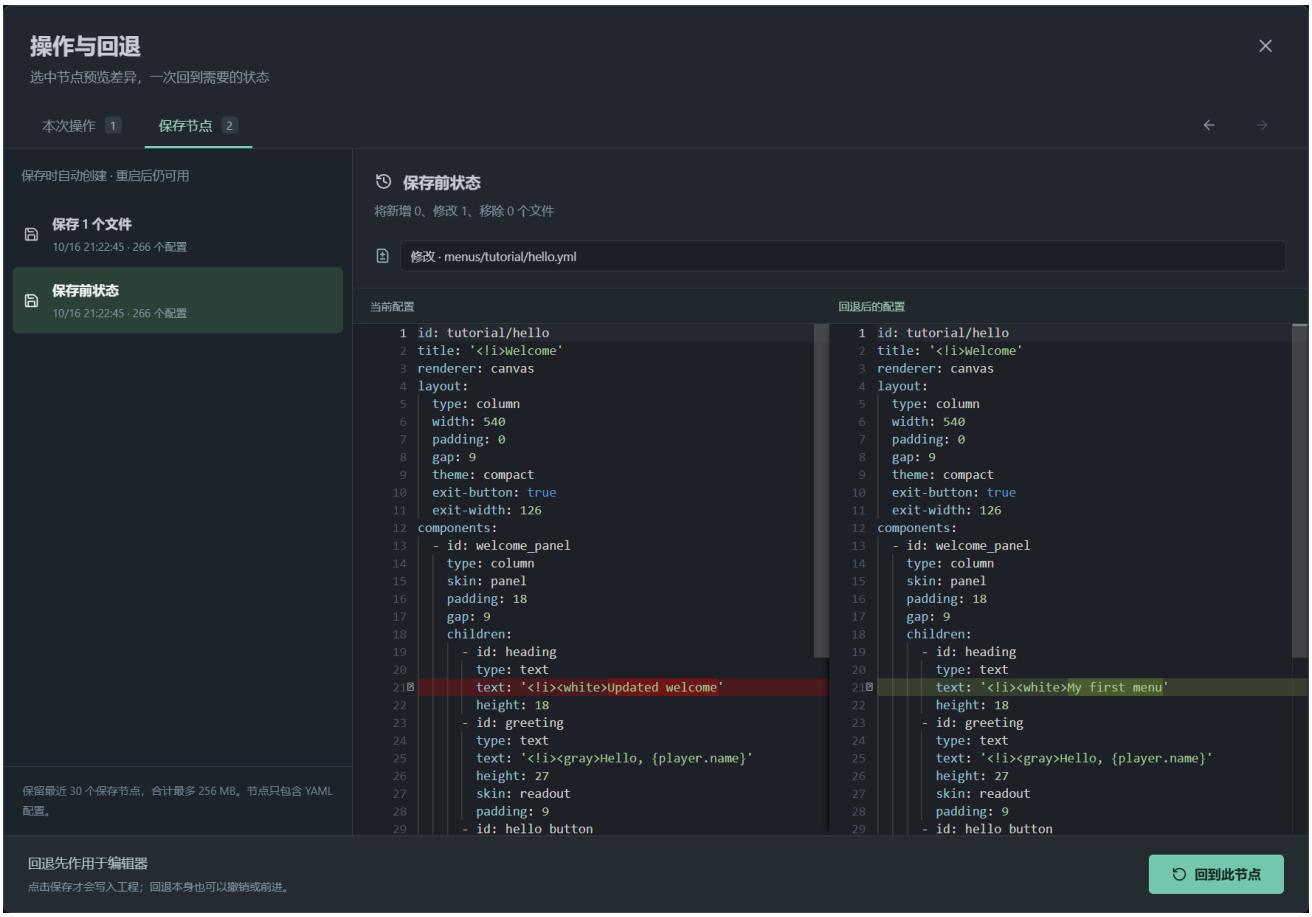
05 Before making another edit, you may move forward to a later node. Editing after moving back creates a new branch and discards the former future operations.

Rule	What it means
Up to 200 nodes	The oldest operations eventually leave the session history.
Typing is grouped	Continuous edits within about 1.2 seconds may share one node.
An import is grouped	One restore can undo several files from the same import.
Reread or reopen	Session operations reset. Use saved points for persistence.

> **Restoring does not mean saving**
> If an edit was already saved, undoing it still requires another save to change disk files. Restoring to before a file was created and then saving removes that file according to the diff; inspect the deletion list first.

12 / RESTORE

Use restore points after reopening Studio



Saved points show cross-file differences before restoring into the editor.

01 After saving configuration, open 操作与回退 > 保存节点 (Operations and restore > Saved points). The first managed save records the before and after states.

02 Select the target time and review the diff plus addition/deletion counts. Click 回到此节点 (Return to this node).

03 Press Esc to return to the workspace, verify the restored result and save. The restore itself becomes an undoable operation, helping recover from a mistaken choice.

04 When moving a project, keep the entire directory including the hidden .futureui-studio folder. Configuration exports do not carry history.

Scope of saved points

Points are stored in .futureui-studio/snapshots, retaining up to 30 recent points and 256 MB compressed total. They cover YAML only, not PNGs, API settings, chats or server player data. Older ordinary backups do not automatically appear as restore points.

13 / AGENT

Connect your own model service

01 Press Ctrl+J to open Agent, then click the Agent 模型设置 (Agent model settings) gear. Confirm the endpoint, protocol, model ID and context limit with your provider.

02 Enter Base URL. OpenAI-compatible services often use an API root ending in /v1. Use your provider's endpoint, not a website's chat page.

03 Choose Chat Completions or Responses to match the service. Changing only the model name does not change the wire protocol.

04 Enter the API key and the exact model ID from the provider. You may type a custom ID directly. 读取模型 (Read models) first saves settings, then tries to fetch the list.

05 Set context capacity, output limit and tool rounds according to the model documentation. Click 保存设置 (Save settings), then return to Agent.

06 Start with a small task: “Read the current menu and explain its buttons without modifying it.” Confirm streaming and tool reads before requesting edits.

Setting	Guidance
Base URL	Use http/https without embedded credentials, query parameters or fragments.
Output parameter (Chat)	Choose max_tokens or max_completion_tokens according to the provider.
Model capabilities	Agent needs tool calls and streaming. Image attachments need a vision-capable model.
Model-list failure	The service may lack a listing endpoint; a valid manually entered ID can still work.

How keys are stored

When enabled, keys are encrypted using the operating system and stored locally. Leaving the key field blank preserves an existing key; use 清除密钥 (Clear key) to delete it. Re-enter it on another PC rather than relying on copied encrypted files.

13 / AGENT

Use the Anthropic Messages protocol

01 In model settings, choose Anthropic · Messages. Enter the matching Base URL, API key and model ID for that service.

02 The standard endpoint may switch automatically to https://api.anthropic.com/v1. For another compatible service, use that service's endpoint.

03 Start with 沿用模型默认行为 (Use model defaults). Select Adaptive or an Extended budget only if the model supports it, with an output budget that meets the service's requirements.

04 Save and try a small read task. If the service rejects an unknown parameter, check thinking-mode compatibility rather than repeatedly increasing the budget.



Actual model connection panel, with no key or specific model entered.

Why a thinking stream may be absent

Studio displays reasoning_content, thinking_delta or a Responses reasoning summary only when the API returns it. Some models return an answer only, which is not necessarily a fault. Studio does not invent private reasoning.

13 / AGENT

Import, update and enable Skills

01 Open 管理 Skill (Manage Skills) in Agent. The built-in futureui-ui supplies plugin capabilities and configuration rules. Keep it, then add your own style or business guidance.

02 Choose 导入目录 (Import directory) or ZIP / SKILL.md. The package should have one unambiguous SKILL.md entry point.

03 Review the name, source and reference files, then enable the Skills for the task. Up to eight enabled Skills are loaded per task.

04 To update, import a Skill with the same name. Studio retains its enabled state and backs up the previous custom version. New content is used from the next task.

05 Use 查看说明 (View instructions) to inspect it. Scripts in a package may be read as reference; importing a Skill does not execute them.

```
---
name: my-menu-style
description: My pixel menu guidelines
version: 1.0.0
---
Use the compact theme and consistent button heights.
Inspect existing configuration before proposing changes.
Read the business rules in references/.
```

Limit	Details
Name	Use letters, digits, hyphens or underscores.
Size	Up to 2 MB per file, 8 MB per package and 500 entries.
Rollback	There is no Skill-version rollback button. Reimport a retained older package when needed.

Skills do not add plugin features

A Skill provides rules and reference material to Agent. It cannot make the plugin support unimplemented fields. When the plugin changes, check the Skill's supported version and configuration contract.

13 / AGENT

Give Agent precise context and review its work

01 Open the target menu and select the component. Use 让 Agent 调整此组件 (Ask Agent to adjust this component) or its canvas context menu to attach the component's source and path.

02 Send files through their context menu or drag them into Agent. Use the send icon or drag on asset cards. Attach a canvas capture for visual feedback. Each task accepts up to eight attachments.

03 Specify the goal, scope and constraints. For example, request spacing and button adjustments on the current page without changing product prices or actions.

04 Watch the streamed response and tool activity. Expand into a floating panel for more room, or stop the task when needed.

05 When Agent proposes changes, inspect the file list, diff and preview. Applying updates the editor; you still need to verify and save.

06 Discard an unsatisfactory proposal and explain why. Apply or discard a pending proposal before starting another generation.

> **A prompt you can copy**
>
> Only edit menus/tutorial/hello.yml. Keep the compact theme and existing actions. Give both buttons equal dimensions and use readout backgrounds for numbers and states. Reuse registered assets; do not invent fields. Inspect the file first and provide a reviewable diff.

Attached text and enabled images are sent to your configured model service. “Reference only” does not send image pixels. A proposal supports up to 24 files and 200 KB per proposed file. If a target changes after generation, application checks for stale content; regenerate or resolve the local edit first.

13 / AGENT

Manage sessions, context usage and compression

01 Click 创建新会话 (Create conversation) for an independent task. Switch through tabs or 会话历史 (Conversation history). Each project retains up to 100 conversations.

02 Stop an active task before archiving or deleting a conversation. Closing its tab can archive it; deletion in conversation history removes the record.

03 Inspect the context-usage ring. Before a request it uses a local estimate; afterward it can use API-reported token usage. It is not a billing amount.

04 Set the real context capacity and compression threshold in model settings. Do not invent a larger capacity merely to hide overflow warnings.

05 Near the threshold, Agent summarizes older context for later requests while retaining the full conversation log. Click the ring to inspect usage and the latest summary. This version has no separate manual-compression button.

Default	Purpose
128,000 context tokens	Initial local budget; adjust to the actual model.
85% compression threshold	Summarizes near this fraction while accounting for output reserve.
8,192 maximum output tokens	Caps output; the reserve is at most about 20% of context.
16 tool rounds	Limits consecutive tool-use rounds within a task.

If compression fails

Keep the original conversation, check the connection and retry; reduce attachments or start a new conversation. Increase capacity only if the model supports it. Compression calls the model service and may consume its quota.

14 / SERVER DEPLOYMENT

Deliver a menu to a Minecraft server

01 Run configuration checks in 发布 (Publish) and resolve errors. Confirm menu IDs, language keys, resources and action references.

02 Save the project and export the needed configuration. For a full first delivery choose all configuration and include assets; for a page update define the exact scope.

03 Back up the server configuration, then copy the matching directories into plugins/FutureUI. Preserve the server's own economy, database and resource-pack distribution settings.

04 Have the server administrator validate and reload, then build and distribute the resource pack through the server's configured process. Image or theme changes require more than YAML reload.

05 In game, inspect buttons, language, window scaling, purchases and dependencies. Integrated third-party menus usually need their original plugin entry point to establish the business session.

Publish action	RCON command
Status	fui status
Validate	fui validate
Reload	fui reload
Build resource pack	fui pack
Open for an online player	fui openfor <player> <menuId>

RCON is an optional command connection

The server must have RCON enabled. Enter host, port (default 25575), password and an online player's name. The password remains in the current panel's memory. RCON does not upload configuration or turn an offline preview into an in-game verification.

15 / UPDATES AND BACKUP

Update after receiving a notification

01	02	03	04	05
Notification	Review version	Download	Save and install	Relaunch

01 By default, the installed app checks about 8 seconds after launch and about every four hours while running. Click 更新 (Updates) to check manually.

02 When a version is available, read its notes. If the app is unfocused and system notifications are allowed, you may also receive a desktop notification.

03 Click Download. Differential downloading is used when possible and may fall back to a full installer. A small download is not guaranteed every time.

04 Stop active Agent work, save the edits you want to keep and click Install update. Studio exits for installation and launches automatically when finished.

05 After relaunch, verify the version and last project. Portable users need to install Setup before using in-app update installation.

What to back up	How
Menus and assets	Copy the whole project, including assets and hidden .futureui-studio.
Delivery configuration	Export a ZIP. It does not include operation history or editor settings.
Model settings	Re-enter the key on another PC and verify protocol, model ID and capacity.

> **Installation remains your choice**
> Finding a version does not install it automatically. Closing the app does not automatically install a downloaded update; use the installation action in the update window.

16 / COMPLETE EXERCISE

Complete an edit-and-deliver exercise

01 Create a practice project and import 01-first-menu. Open tutorial/hello and save. The menu initially has one message button.

02 Set hello_button height to 36 and change its label. Inspect its hovered appearance.

03 Import 02-merge-update with Merge. Use the incoming heading but keep the current hello_button.text. Confirm that the new Close button appears.

04 Import 03-language-entries and set greeting.text to '@tutorial.greeting'. Switch between Chinese and English and inspect text lengths.

05 Save all files. Open Saved points and inspect the before/after diff without restoring yet.

06 Export the current menu YAML outside the project. Then export the menu plus both language files as ZIP. For a full theme delivery, separately export all configuration and assets.

07 Move back to an earlier session node, inspect the canvas and move forward to the latest node. Do not save the restored state before confirming it.

08 Optional: ask Agent to refine only this page's spacing while retaining actions and product data. Review, apply and save, then verify on a server.

> **Completion checkpoint**
> You should be able to explain each file, preview both languages, distinguish saving from exporting and recover an edit through history. Finish this exercise before editing production menus.

The samples are separated by step, and both hello.yml files target the same path. Import the corresponding folders separately rather than mixing every YAML from the whole tutorial ZIP in one import.

17 / TROUBLESHOOTING

Editor, import and restore troubleshooting

Symptom	Check in this order
Blank canvas	Did you open a menu rather than a language file? Check YAML errors and visibility conditions, click Fit and reset the workspace layout.
Blurry text or images	Inspect zoom, source dimensions and pixel edges. Preview is not the final game-font/shader renderer.
Old values remain after import	Merge may default to keeping current values. Expand conflicts, choose incoming values and apply.
Images missing after import	Import reads YAML only. Check assets, image IDs and definitions, then supply missing PNGs.
Save blocked by external edits	Export current edits outside the project first. Reread disk files and merge the needed changes rather than overwriting another edit.
An old state is absent	Session history resets on reopen. Saved points cover managed saves made after the feature became available.
Disk unchanged after restore	Restore updates the editor first. Save after confirming it.
ZIP did not remove old files	Exports do not include a deletion manifest. The recipient must handle removals explicitly.

When the diff is unclear

Export the current version as a copy, then compare files individually. Keep the original project and restore points instead of repeatedly overwriting files to dismiss a warning.

17 / TROUBLESHOOTING

Agent, updates and server connection issues

Symptom	What to do
401 / 403	Verify the API key, service endpoint and account permissions. A website session token is not necessarily an API key.
404 or unsupported endpoint	Check Base URL and protocol. Chat, Responses and Messages use different endpoints.
Model listing fails	Manually enter a provider-confirmed model ID. The service may not implement listing.
No streaming or tool activity	Verify streaming and tool-call support. Basic chat compatibility does not guarantee full Agent capability.
No thinking content	The API may not return it. Inspect answers and tool results instead of treating its absence as failure.
Context overflow / compression failure	Check the true capacity, reduce attachments, retry compression or start another conversation.
Proposal is stale	The file changed after generation. Review local edits and regenerate the proposal.
Update installation unavailable	Use the installed build. Development and portable builds do not install updates directly. Check GitHub connectivity.
RCON connection fails	Check server enablement, host, port, password and network access. Menu preview itself does not need RCON.

For quota or parameter errors, use the provider's error message and current model documentation. Do not paste real keys into public issues or screenshots. Include the version, reproduction steps, credential-free error message and a minimal configuration.

18 / QUICK REFERENCE

Keyboard shortcuts and further help

Shortcut	Action
Ctrl+O	Open a project
Ctrl+S	Save all configuration edits
Ctrl+Shift+I	Import and merge
Ctrl+Shift+E	Quick export
Ctrl+Alt+H	Operations and restore
Ctrl+J	Open or focus the Agent sidebar
Ctrl+P	Command palette
Ctrl+Z / Ctrl+Shift+Z	Undo / redo
Ctrl+D / Delete	Duplicate / delete selection outside text inputs
F	Focus / restore canvas outside text inputs
F5 / Ctrl+R	Reread project files from disk, not server reload
Esc	Dismiss a dialog or command palette

Mouse: Ctrl+wheel zooms; use the pan tool or middle-button drag to pan; double-click an asset to insert or replace an image; drag layers to reorder. Text inputs and the source editor may handle their own shortcuts first.

> **Include this in a bug report**
> Studio version, operating system, minimal steps, expected versus actual result, relevant YAML and screenshots. For in-game issues also include server and client versions.

Downloads and updates: GitHub Releases
Project and tutorials: FutureUI Studio
Report a problem: GitHub Issues